End-to-End CI/CD Pipeline with Helm, ArgoCD, and GitHub Actions

A GitOps-Driven Approach to Cloud-Native Application Delivery

Ayomide Ajayi

Larry.ayomide@techbleat.co.uk

1.0 Project Overview

This project implements the **automated deployment** of a **containerized web application** on a **Kubernetes cluster** using **Helm charts** and **GitOps principles.** A **four-node Kubernetes cluster** was created using **KIND (Kubernetes IN Docker) on Docker Desktop** to simulate a production-like environment.

The deployment process involved **building and containerizing the application** using a **Dockerfile**, then **pushing the image to a local container registry** for efficient image storage and access. **Helm charts** were configured to define the application's Kubernetes resources, ensuring a standardized, reproducible, and version-controlled deployment.

To achieve **continuous integration and deployment (CI/CD), GitHub Actions** (with a self-hosted runner) was used to automate the Helm-based deployment pipeline—triggering builds and releases on code changes. **ArgoCD** was then integrated to **manage application deployments declaratively** using **GitOps.** It **continuously monitored** the **Helm chart repository** and **synchronized** the cluster state with the desired configuration stored in Git, **enabling automated rollbacks, drift detection, and visual insight** into the deployment lifecycle. Finally, **port-forwarding** was used to enable local access to the deployed application, providing interactive access to the running service within the cluster.

This project demonstrates a **scalable, automated, and GitOps-driven approach** to **deploying containerized applications in Kubernetes**—leveraging **Helm for Kubernetes packaging, GitHub Actions for CI/CD orchestration, and ArgoCD for declarative and auditable GitOps delivery.**

2.0 Technologies Used

Docker

Docker is a **containerization platform** that enables applications to be packaged into **lightweight, portable containers**. These containers encapsulate all dependencies, ensuring consistency across different environments. Docker allows services to run in **isolated environments**, either on the same host or distributed across multiple hosts, improving scalability and resource efficiency.

Kubernetes

Kubernetes is an **open-source container orchestration platform** designed to **deploy, manage, and scale** containerized applications. It automates key processes such as **service discovery, load balancing, and self-healing** while ensuring high availability and fault tolerance. Kubernetes also supports **auto-scaling**, enabling applications to dynamically adjust resources based on demand.

Helm

Helm is a **package manager for Kubernetes** that simplifies application deployment by bundling multiple Kubernetes resource files into a **single package**, known as a **Helm chart**. Helm allows applications to be installed with a **single command**, providing a default configuration while enabling customization through **values files**. This streamlines deployment, version control, and rollback processes, making Kubernetes application management more efficient.

3.0 Why Use Helm Charts?

Using **Helm charts** for deploying a web application on Kubernetes is far more **scalable, reusable, and maintainable** than using raw manifest files. The following presents the advantages of using Helm Charts over raw manifest files:

1. Parameterization & Configurability

- Helm allows you to define **values.yaml** files, making deployments **customizable** without modifying core Kubernetes manifests.
- Raw YAML files require manual edits, leading to inconsistent configurations across environments.

2. Versioning & Rollbacks

- Helm tracks releases, making it easy to **rollback** to a previous state:
  e.g helm rollback coweb 2
- However, with raw manifests, reverting requires manually undoing changes or maintaining backups.

3. Dependency Management

- Helm **bundles dependencies** into charts (charts/ directory), handling things like database deployments within the same package.
- With raw manifests, you need to manage external dependencies separately.

4. Simplified Deployment with One Command

- Instead of applying multiple YAML files manually, Helm deploys everything in one go: helm upgrade --install coweb ./coweb --namespace coweb-ns
- Unlike raw manifests that require multiple kubectl apply commands, increasing complexity.

5. Dry-Run & Pre-Validation

- Helm supports **template rendering & dry run**:

  helm template ./coweb

  helm upgrade --install coweb ./coweb --dry-run --debug
- Raw manifests lack built-in validation beyond Kubernetes' basic syntactic checks.

6. Built-in Templating & Reusability

- Helm templates enable reuse across different and multiple environments (dev, staging, prod) without file duplications.
- Raw manifests require multiple static YAML files for each environment, increasing maintenance effort.

When to Use Raw Manifests Instead

- For small, **single-use, manual deployments**.
- When dealing with **custom CRDs** not easily managed via Helm.

In summary, Helm streamlines deployments, reduces human error, and efficiently scales Kubernetes applications. Its ability to ensure repeatability and flexibility makes it a practical solution for managing production-grade web applications with reliability and ease.

4.0 Project Implementation

4.1 Prerequisites

- **Docker Desktop** installed and running.
- **Kubernetes enabled** within Docker Desktop.
- **Helm installed** on your local machine.
- **GitHub repository** with the application source code.
- **Self-hosted GitHub Actions runner** configured on the local machine.
- **ArgoCD manually configured (service exposed)** on the **local K8S cluster**
- **Basic knowledge** of Docker, Kubernetes, Helm, and GitHub Actions.

4.2 Step-by-Step Implementation

Follow this **structured approach** to implement **CI/CD** for deploying the **containerized web application** on a **Kubernetes cluster running on Docker Desktop**, using **Helm and GitHub Actions**. This workflow ensures **continuous integration and deployment (CI/CD)** by automating the **build, testing, and deployment** process using dynamic and custom-built **Kubernetes manifests**, enabling a streamlined and repeatable deployment pipeline.

Step 1: Containerize and Push Application

- **Create a Dockerfile** for the web application:
- **Build the Docker image**: docker build -t coweb .
- **Push the image to Docker Hub**: After tagging

Step 2: Create a Helm Chart

- Run the following command to generate a new Helm chart structure:

helm create coweb

This creates a directory named coweb that contains default templates for deployments, services, and configurations.

Step 3: Define the Deployment

- Modify the Helm **Deployment template** (coweb/templates/deployment.yaml)
- Uses Helm template variables ({{ .Values.image.repository }}) to dynamically set the **Docker image**.
- Ensures **six replicas** of the web application are running.

Step 4: Define the Service

- Modify the Helm **Service template** (coweb/templates/service.yaml)
- Exposes the web application on **port 80** using Kubernetes **NodePort**.

Step 5: Configure Helm Values

- Modify the default values in my-web-app/values.yaml
- This ensures Helm dynamically sets the **Docker image repository and tag**.

Step 6: Deploy the Helm Chart

- Run the following command to install the Helm chart:

  helm install coweb ./coweb --namespace coweb-ns
- Verify that the deployment and service are running:

  kubectl get pods -n coweb-ns

  kubectl get svc -n coweb-ns

Step 7: Automate CI/CD with GitHub Actions

- **Configure the Self-Hosted Runner and start the runner**: .\rum.cmd or ./run.sh

Step 8: Create GitHub Actions Workflow

- **Create a .github/workflows/deploy.yml file** in the repository.
- **Define the workflow**

Step 9: Configure GitHub Secrets: if NOT using self-hosted runner

Step 10: Trigger Deployment: Push changes to the repository

Step 11: Access the Application

- **Use port-forwarding to access the application**:

  kubectl port-forward svc/coweb-service 31555:80 -n coweb-ns
- **Open the application in a browser**:

  http://localhost:31555

4.3 Key Project Highlights

Helm Linting Workflow

The helm lint ./coweb command validates the Helm chart in the coweb directory, checking Chart.yaml, values.yaml, and templates for syntax errors, missing values, and adherence to best practices. Helm processes templates to ensure correct references to defined values. If misconfigurations such as undefined .Values references, incorrect field types, or structural issues are detected, it generates warnings or errors that may lead to deployment failures. Linting helps maintain Helm charts by ensuring consistency across environments and preventing runtime errors. As a key step in CI/CD pipelines, it catches issues before deployment, reducing risks and improving reliability.

CodeRabbit

CodeRabbit is an AI-powered code review tool that integrates with GitHub to automate pull request reviews. Installed in VS Code and triggered on push, it analyzes code changes, summarizes identified issues, and suggests improvements using AI models. It can also be integrated into CI/CD pipelines by authorizing CodeRabbit on GitHub, configuring it for the repository, and enabling it in GitHub Actions. This provides several benefits, including reducing manual code reviews, AI-powered issue detection, continuous feedback on commits within pull requests, and customizable rules via .coderabbit.yaml.

Helm Rollbacks & Kubernetes Health Checks

Helm rollbacks ensure application stability by reverting deployments to previous functional versions in case of failures. Users can restore an earlier release by specifying the release name and revision. Tracking rollback history improves transparency, helping teams understand deployment evolution.

Kubernetes health checks enhance application reliability through liveness and readiness probes. Liveness probes restart unhealthy containers, while readiness probes prevent unready instances from receiving traffic. Configuring these probes in deployment.yaml improves uptime and mitigates cascading failures.

Integrating Helm rollbacks with Kubernetes health checks strengthens deployment resilience, minimizing downtime and enhancing fault tolerance. Automating rollback triggers based on failed health checks ensures rapid recovery. Teams can further optimize workflows by incorporating Kubernetes StatefulSets for consistent recovery strategies.

CI/CD Integration with Helm

Integrating Helm chart deployment with CI/CD automation using GitHub Actions streamlines updates. Changes pushed to a repository trigger automatic Helm deployments while validating configurations through linting to catch errors early. In case of deployment failures, Helm's rollback mechanism restores the previous stable version, preventing disruptions. Additionally, Helm's upgrade strategy enables zero-downtime updates, maintaining application availability.

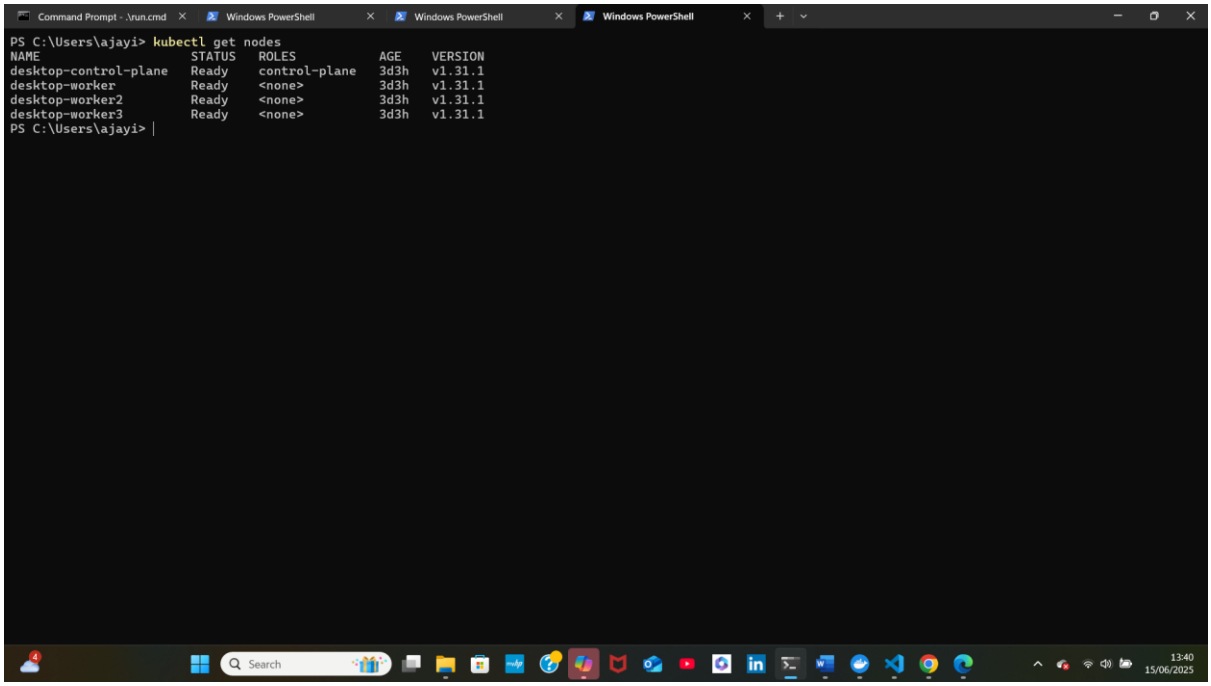
ArgoCD

The deployment of the containerised web application using Helm Kubernetes cluster was also integrated with ArgoCD (which was installed manually on the local machine) to enable GitOps-style continuous delivery once the GitHub Actions workflow is triggered.

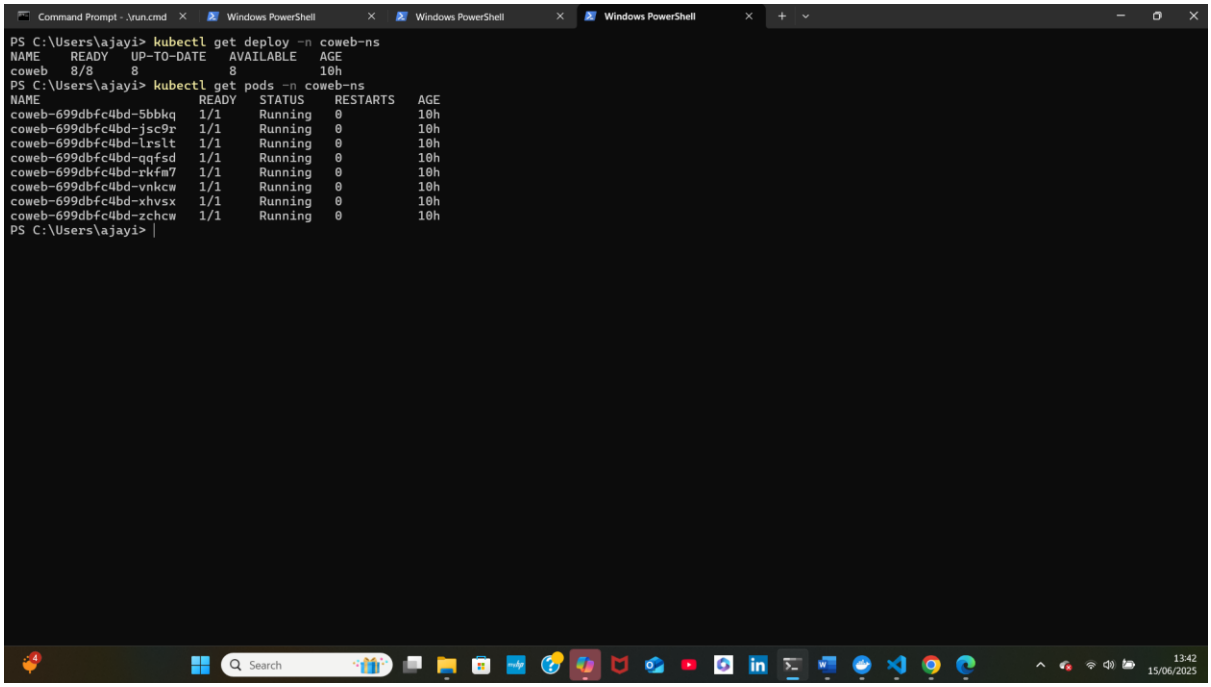
By combining **Helm rollbacks**, **Kubernetes health checks**, **CI/CD automation**, **ArgoCD's declarative GitOps synchronization** and **AI-powered code reviews**, deployments become more resilient and self-healing. ArgoCD continuously monitors the desired state stored in Git, ensuring automatic remediation and rollback in the event of discrepancies or failures. This reduces manual intervention while accelerating recovery, making it ideal for production environments where **uptime, stability, and automated deployment integrity** are critical.

5.0 Project Artefacts

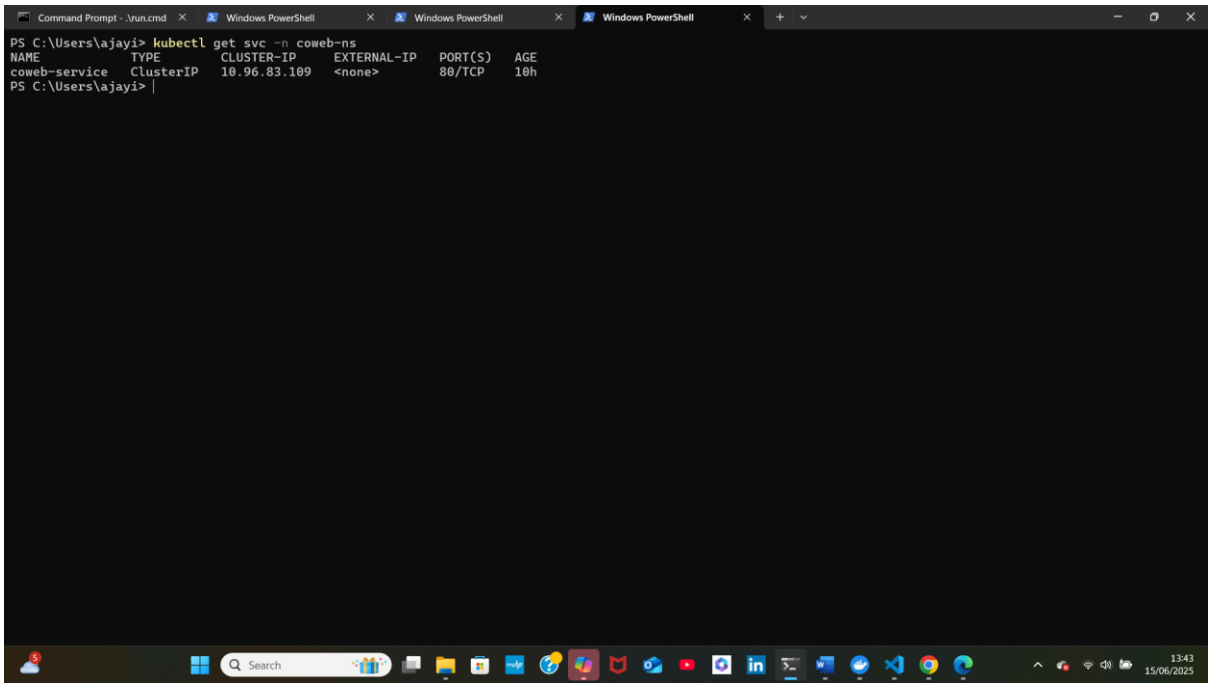
Four-Node Kubernetes Cluster in KIND



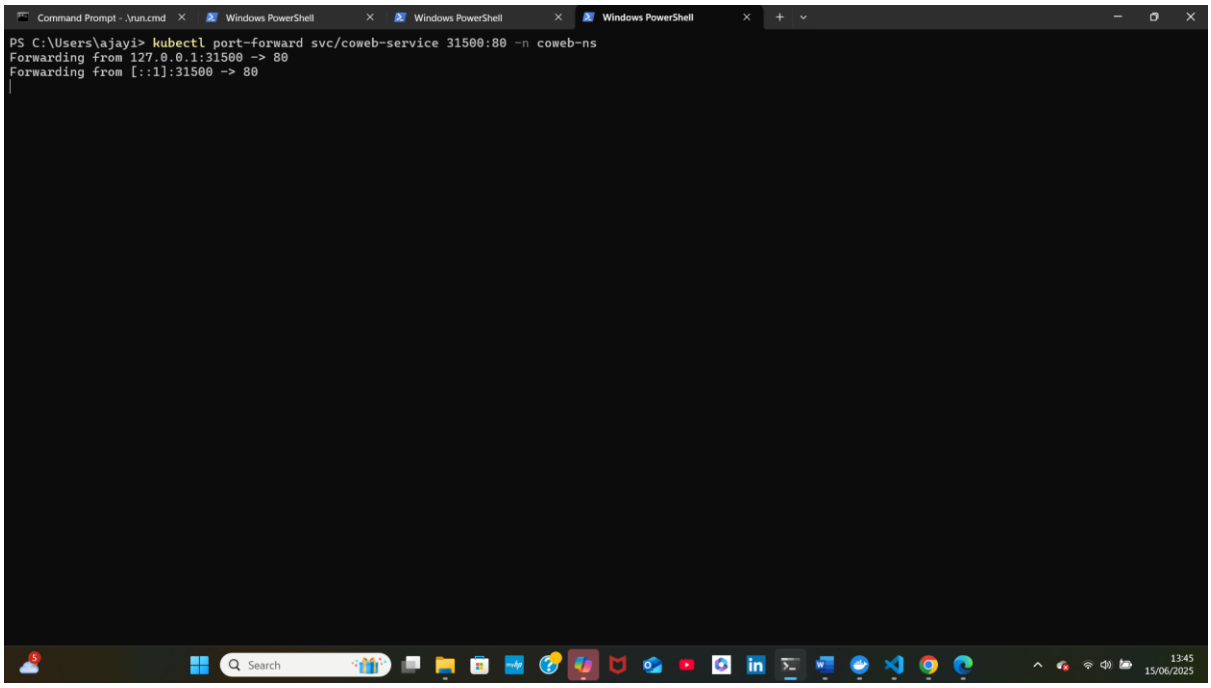
Web Application Deployment with 8 Replicas



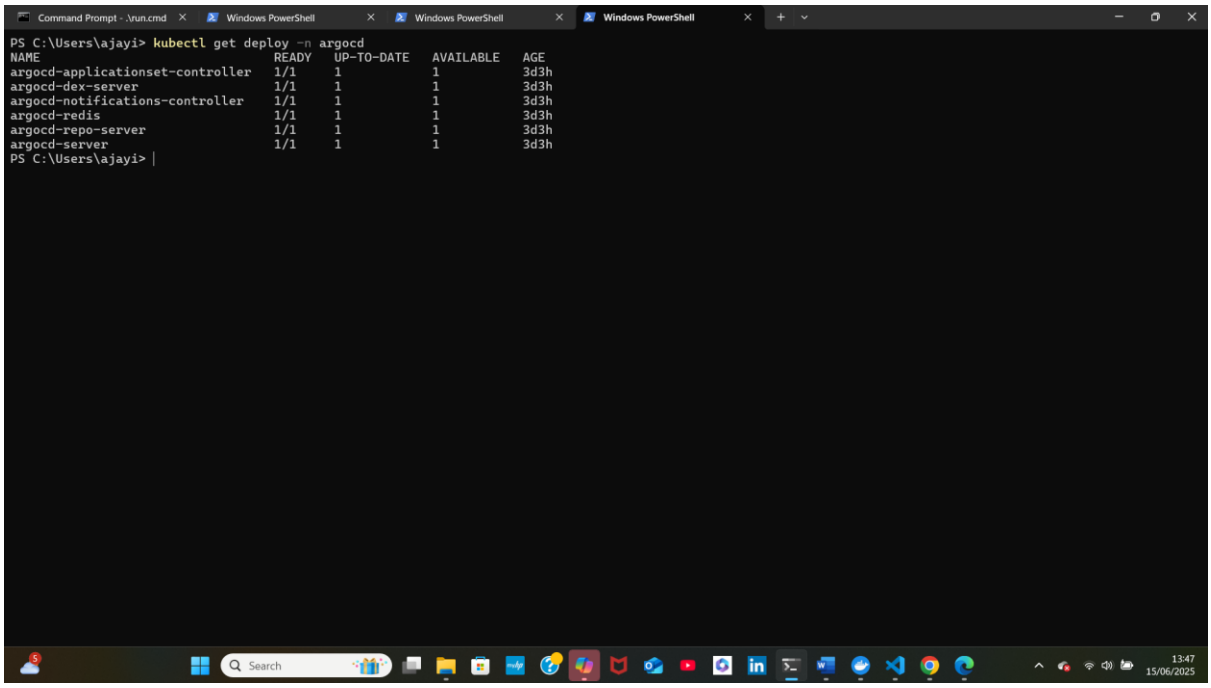
Web Application Service



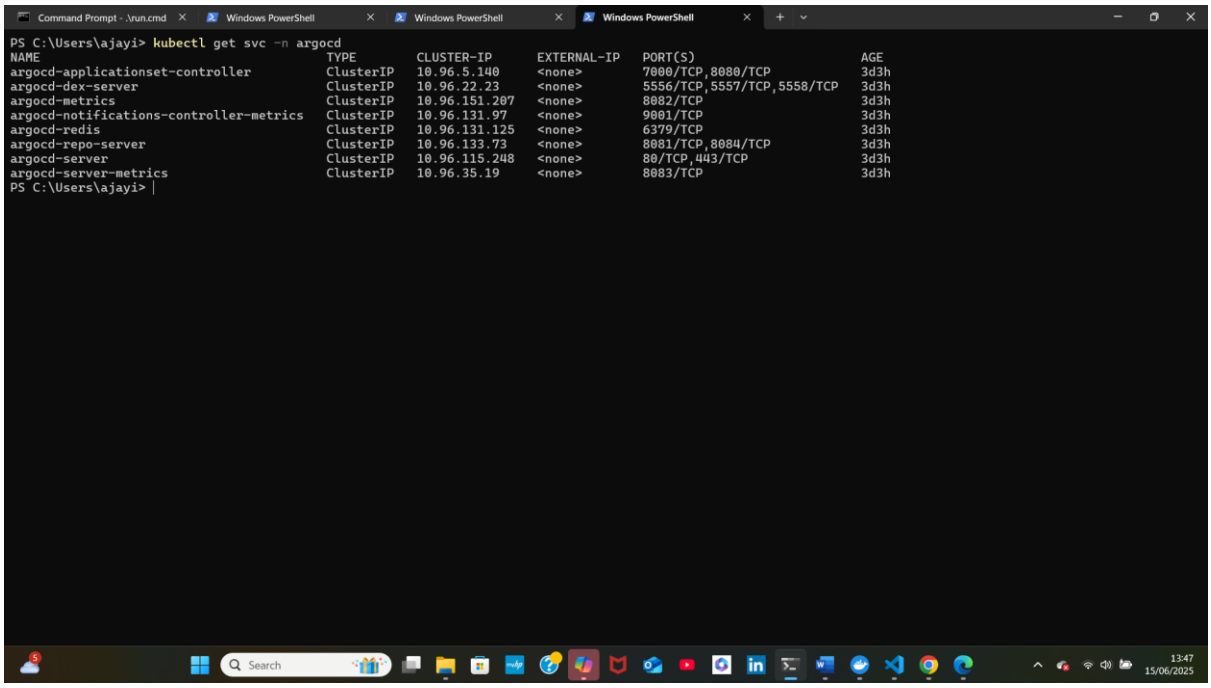
SVC Port-Forwarding



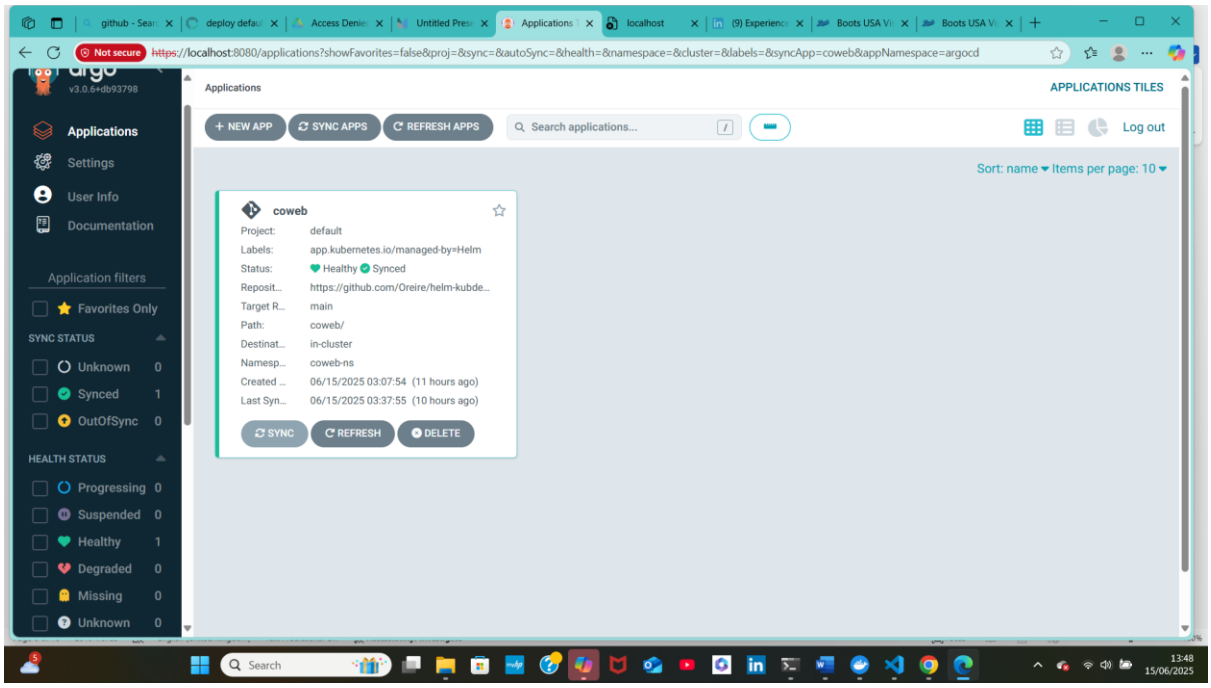
ArgoCD Deployment



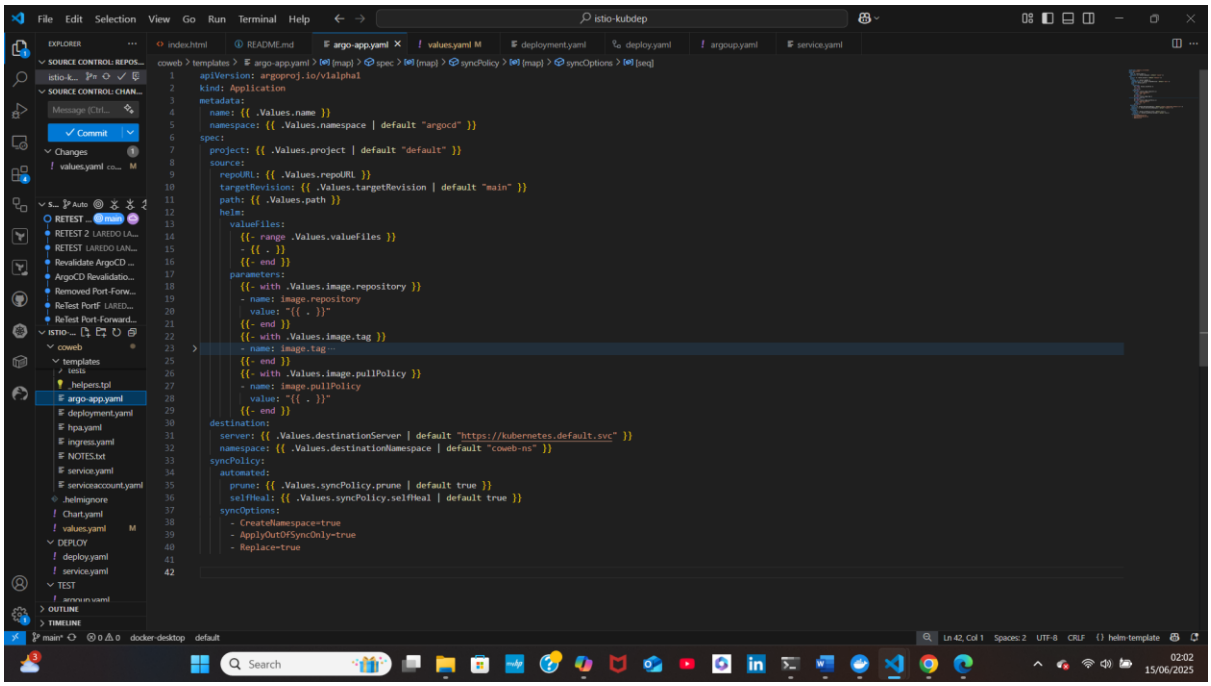
ArgoCD SVC



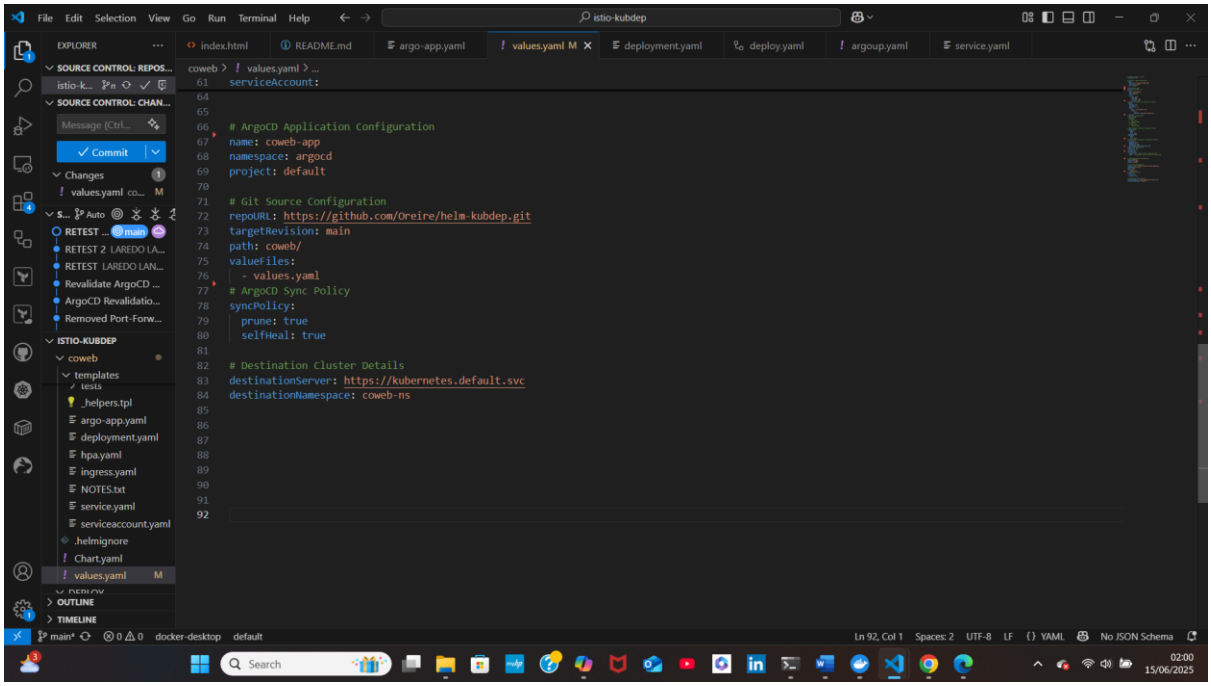
ArgoCD UI



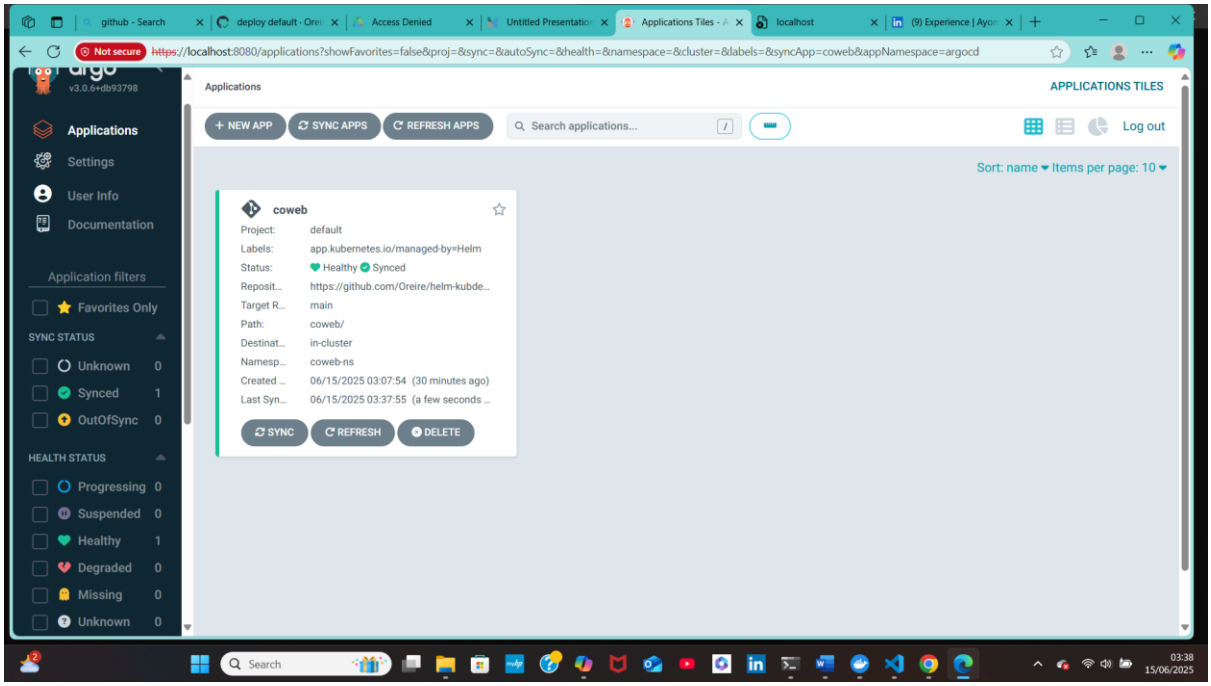
ArgoCD argo-app.yaml file



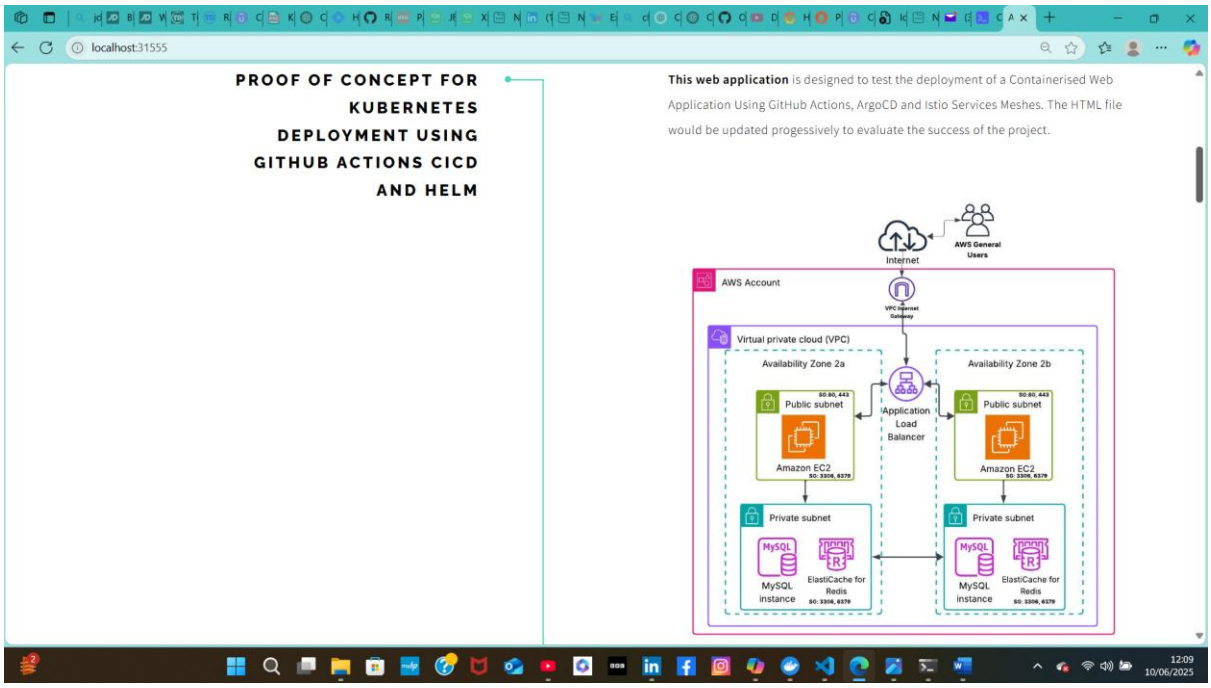
ArgoCD Helm values.yaml

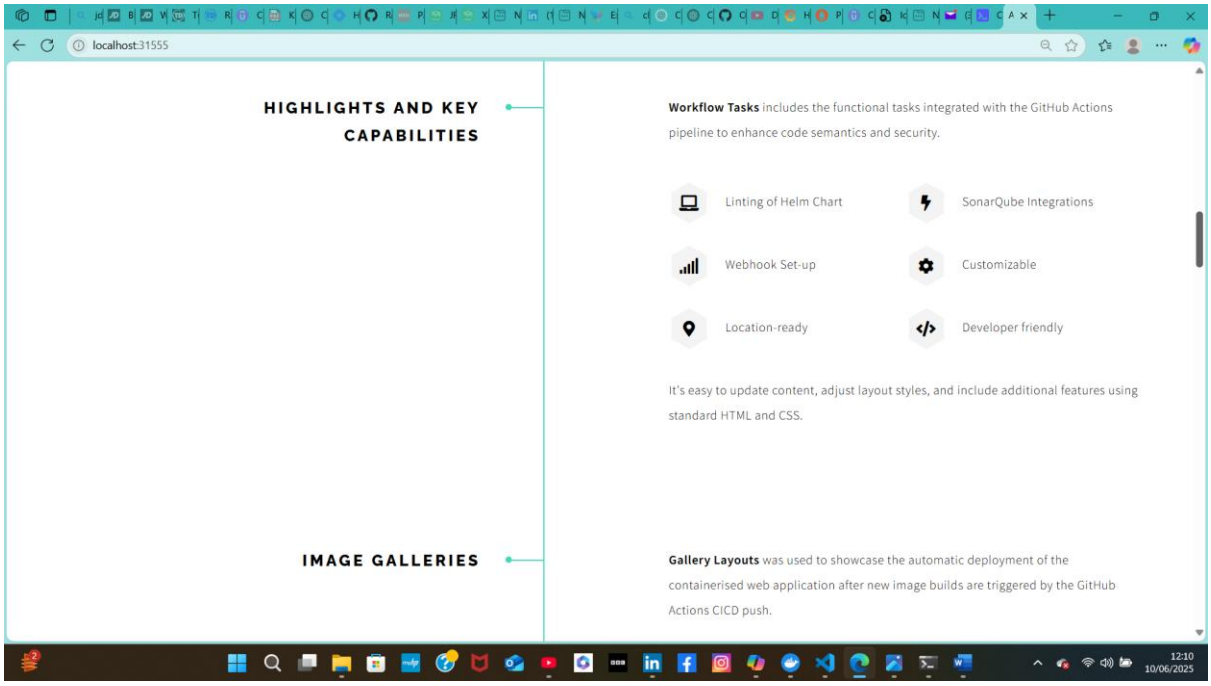


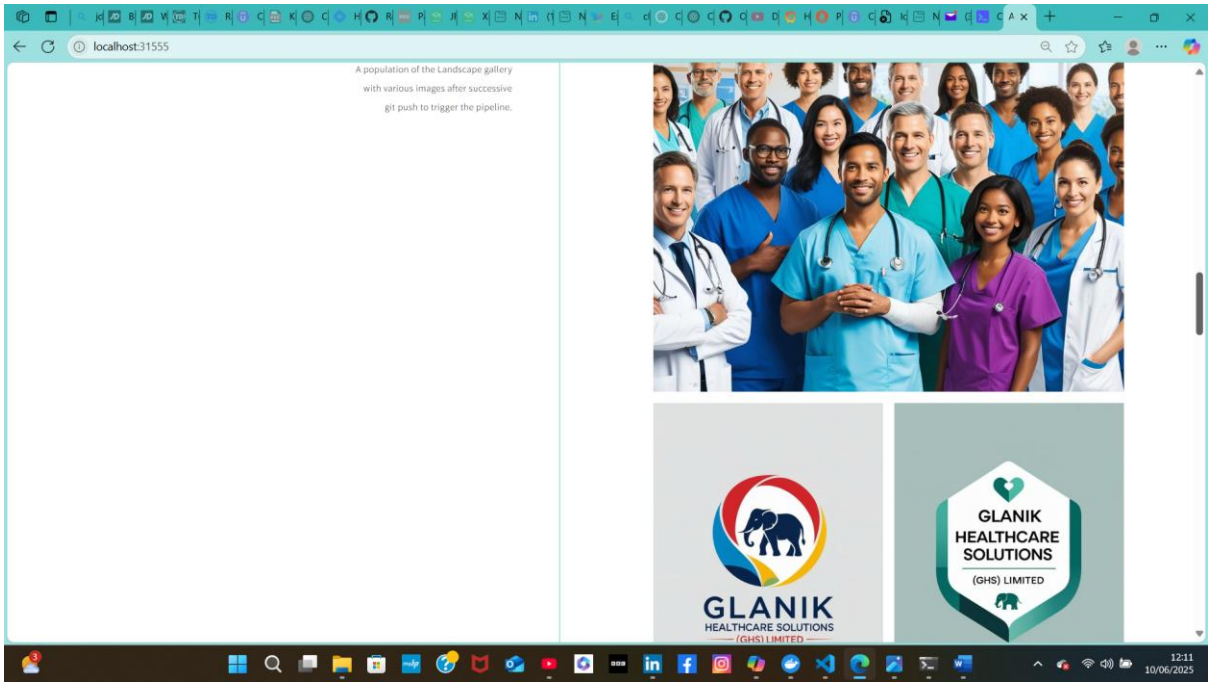
ArgoCD Sync UI

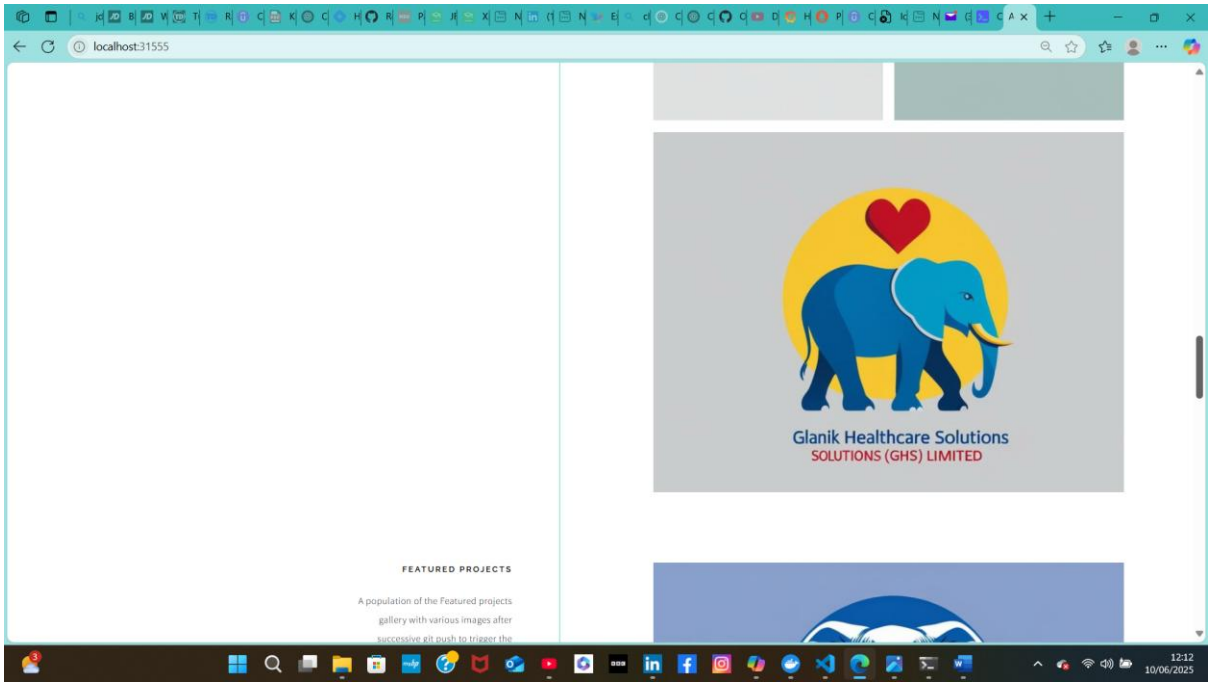


Web Application Deployed

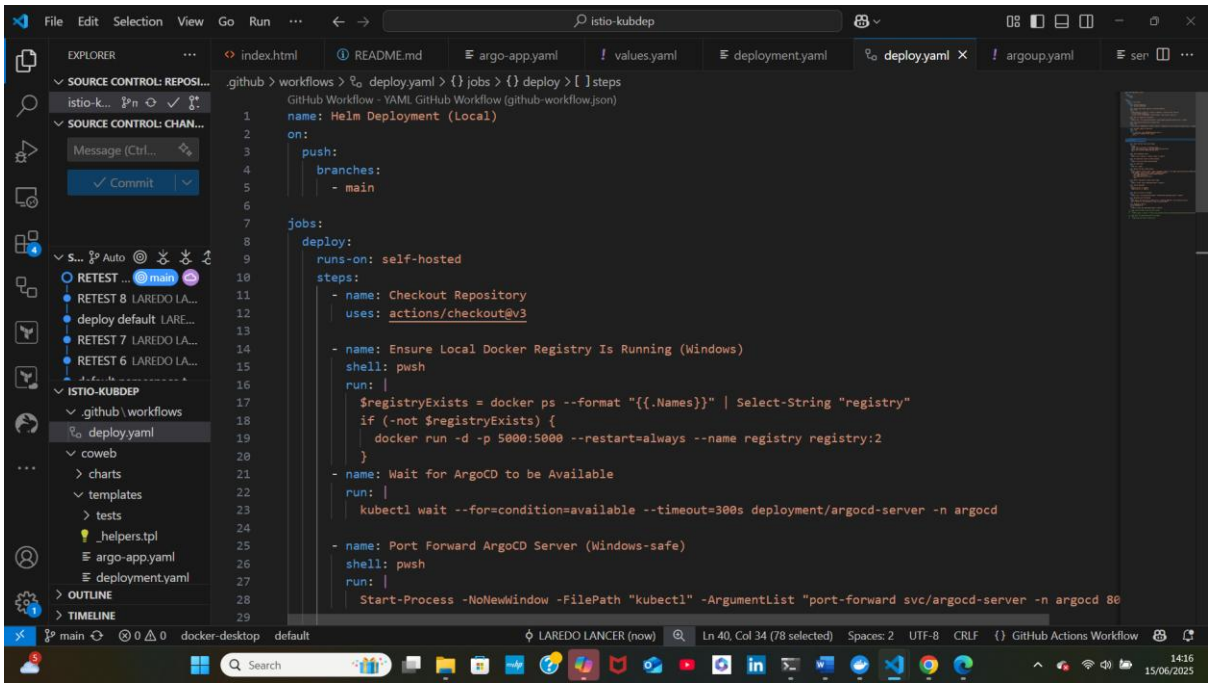




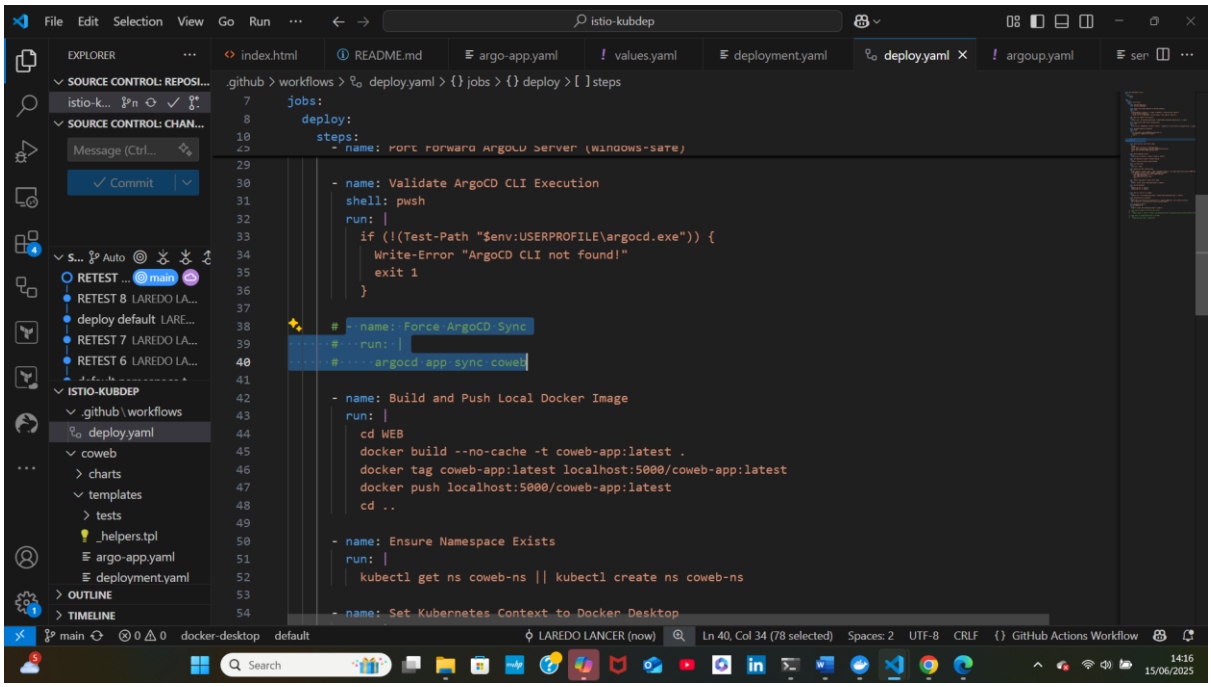

localhost:31555
A population of the Landscape gallery
with various images after successive
git push to trigger the pipeline.
GLANIK
HEALTHCARE SOLUTIONS
(GHS) LIMITED
GLANIK
HEALTHCARE
SOLUTIONS
(GHS) LIMITED
12:11
10/06/2025


localhost:31555
Glanik Healthcare Solutions
SOLUTIONS (GHS) LIMITED
FEATURED PROJECTS
A population of the Featured projects
gallery with various images after
successive git push to trigger the
12:12
10/06/2025

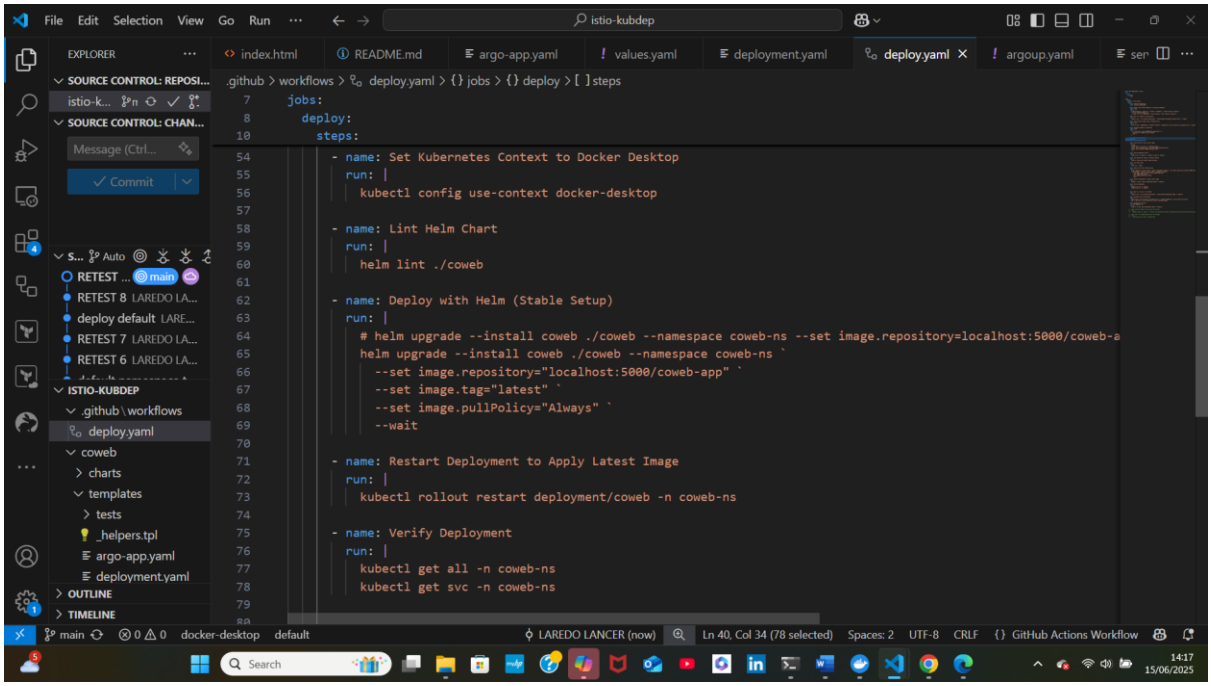
GitHub Actions Workflow

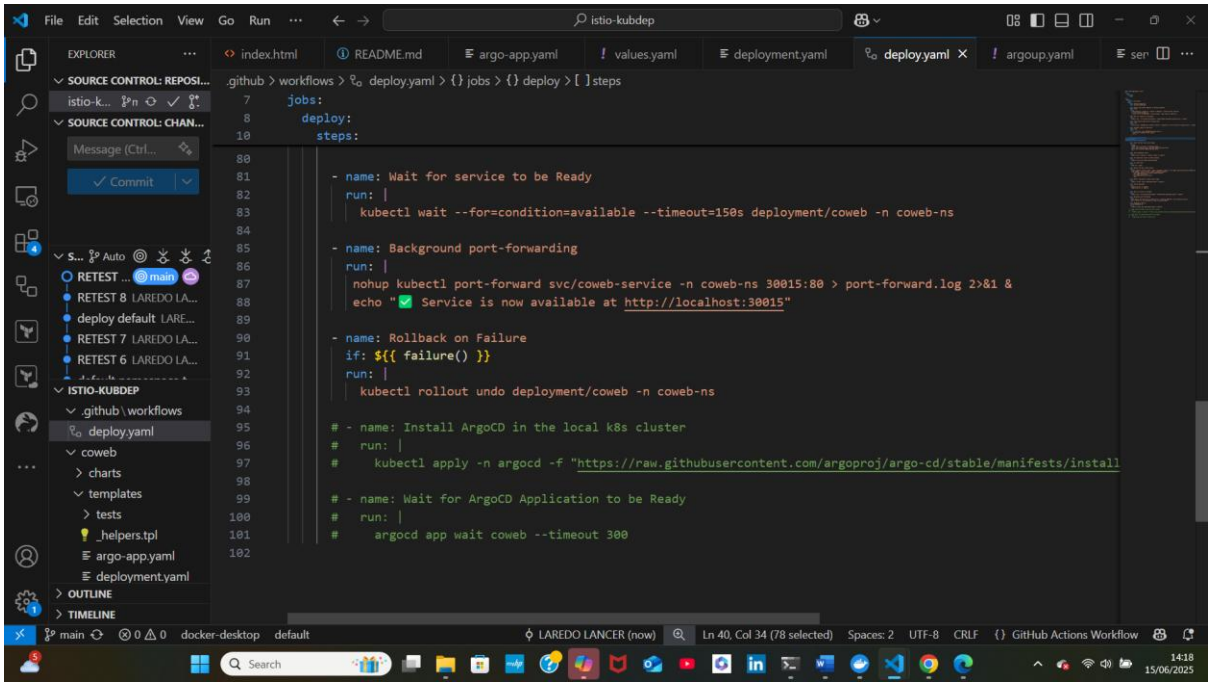


```yaml
name: Helm Deployment (Local)
on:
  push:
    branches:
      - main

jobs:
  deploy:
    runs-on: self-hosted
    steps:
      - name: Checkout Repository
        uses: actions/checkout@v3

      - name: Ensure Local Docker Registry Is Running (Windows)
        shell: pwsh
        run: |
          $registryExists = docker ps --format "{{.Names}}" | Select-String "registry"
          if (-not $registryExists) {
            docker run -d -p 5000:5000 --restart=always --name registry registry:2
          }
      - name: Wait for ArgoCD to be Available
        run: |
          kubectl wait --for=condition=available --timeout=300s deployment/argocd-server -n argocd

      - name: Port Forward ArgoCD Server (Windows-safe)
        shell: pwsh
        run: |
          Start-Process -NoNewWindow -FilePath "kubectl" -ArgumentList "port-forward svc/argocd-server -n argocd 80
```

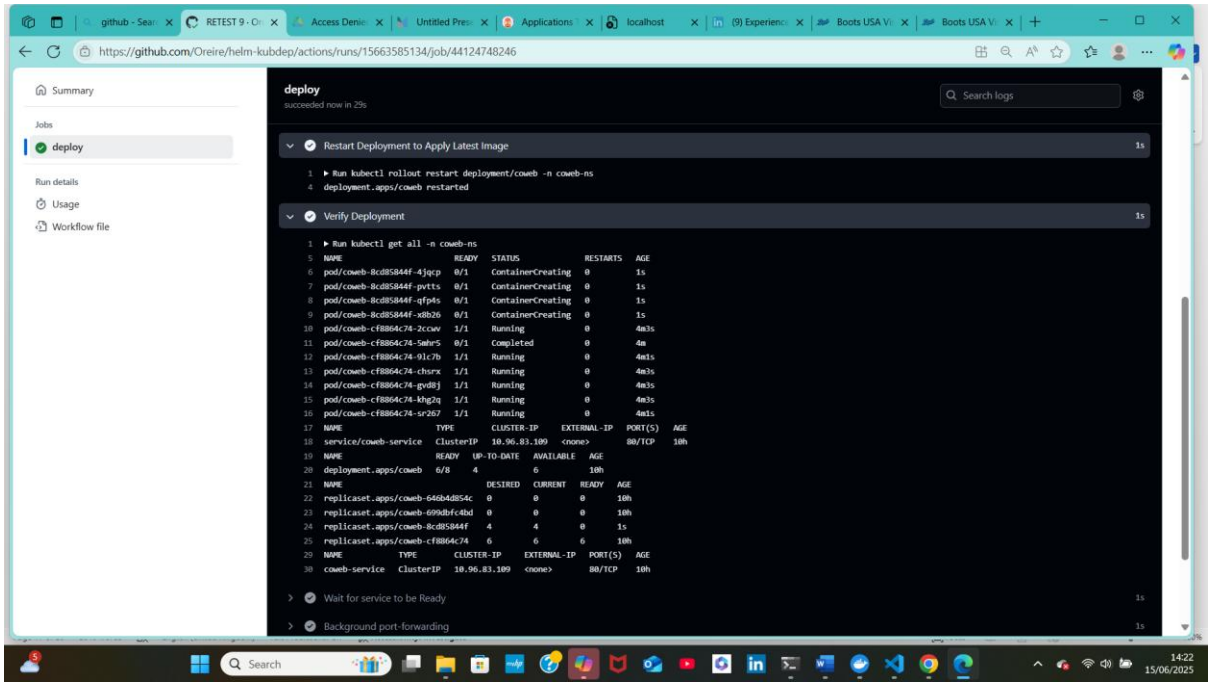


```yaml
jobs:
  deploy:
    steps:
      - name: Port Forward ArgoCD Server (Windows-safe)

      - name: Validate ArgoCD CLI Execution
        shell: pwsh
        run: |
          if (!(Test-Path "$env:USERPROFILE\argocd.exe")) {
            Write-Error "ArgoCD CLI not found!"
            exit 1
          }

      # - name: Force ArgoCD Sync
      #   run: |
      #     argocd app sync coweb

      - name: Build and Push Local Docker Image
        run: |
          cd WEB
          docker build --no-cache -t coweb-app:latest .
          docker tag coweb-app:latest localhost:5000/coweb-app:latest
          docker push localhost:5000/coweb-app:latest
          cd ..

      - name: Ensure Namespace Exists
        run: |
          kubectl get ns coweb-ns || kubectl create ns coweb-ns

      - name: Set Kubernetes Context to Docker Desktop
```

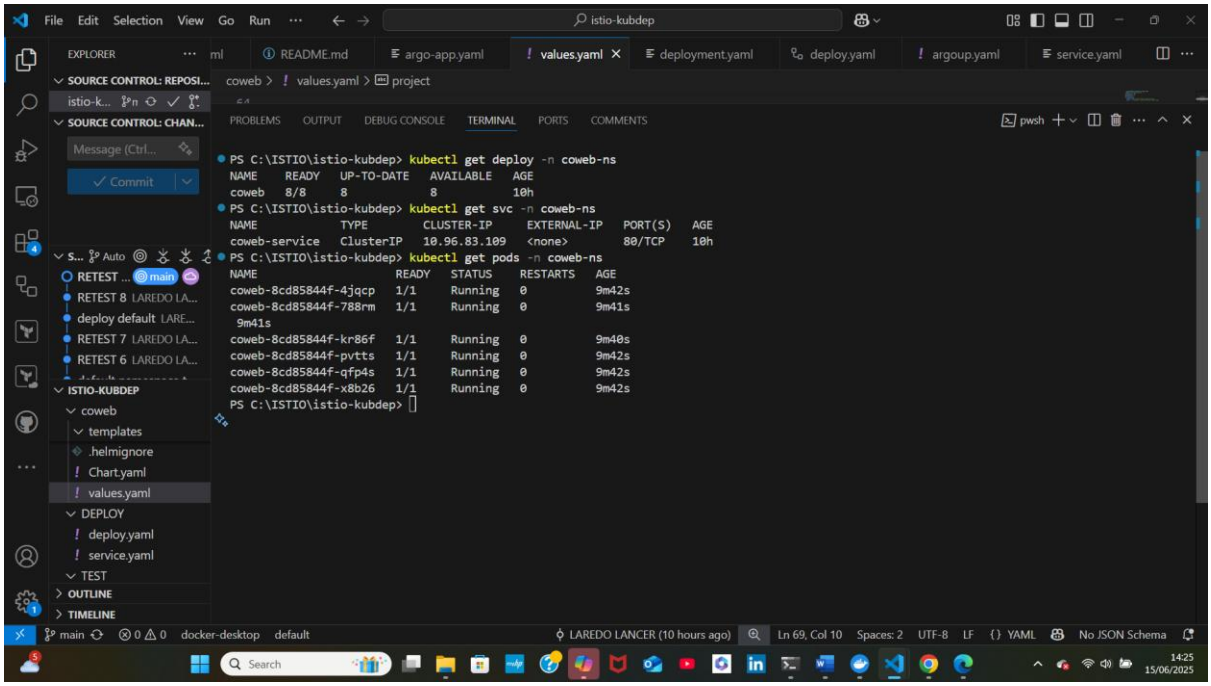




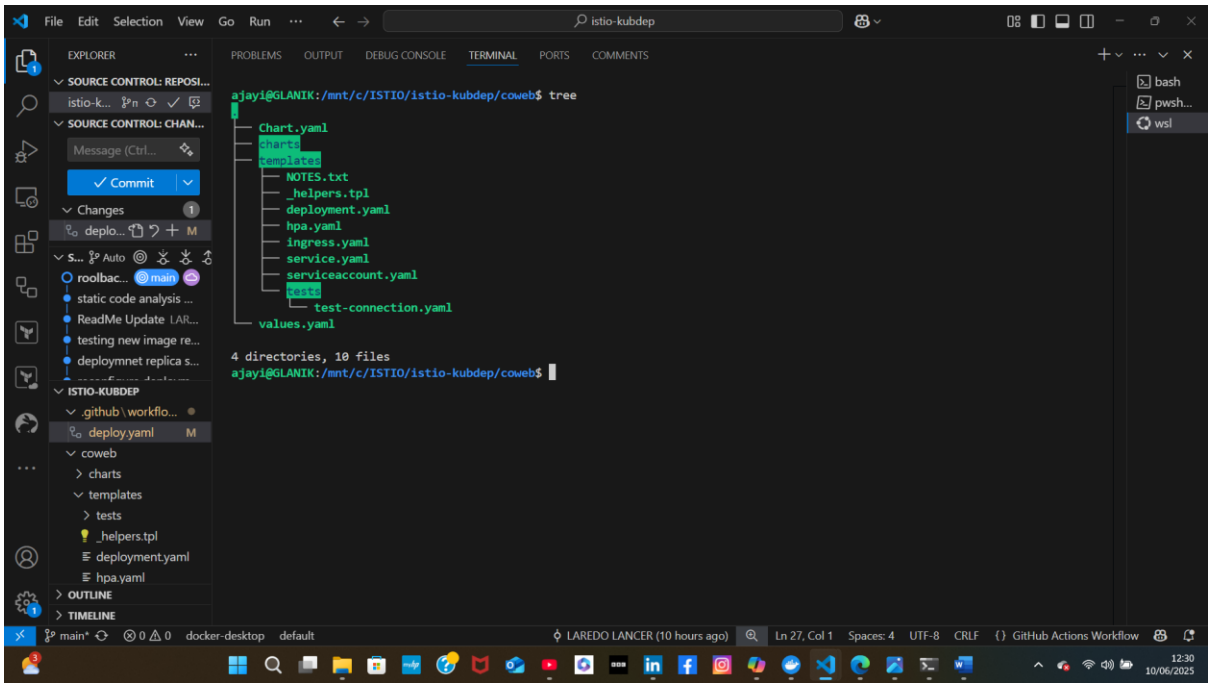
GitHub Actions Workflow Success

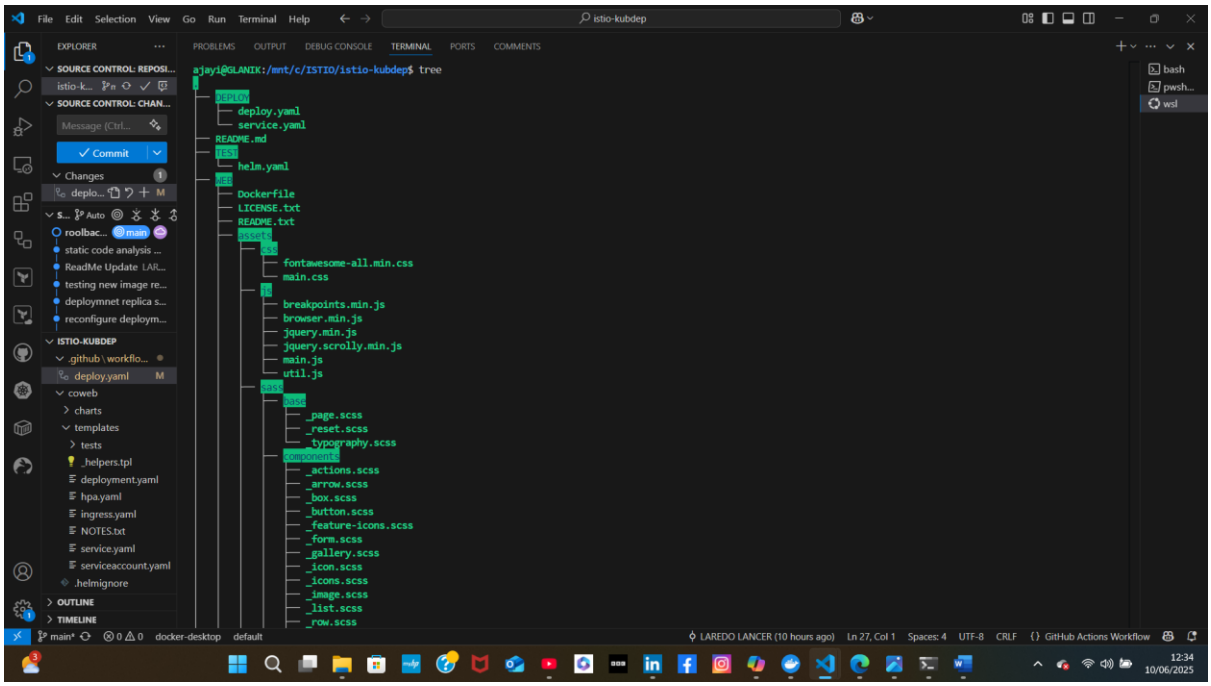


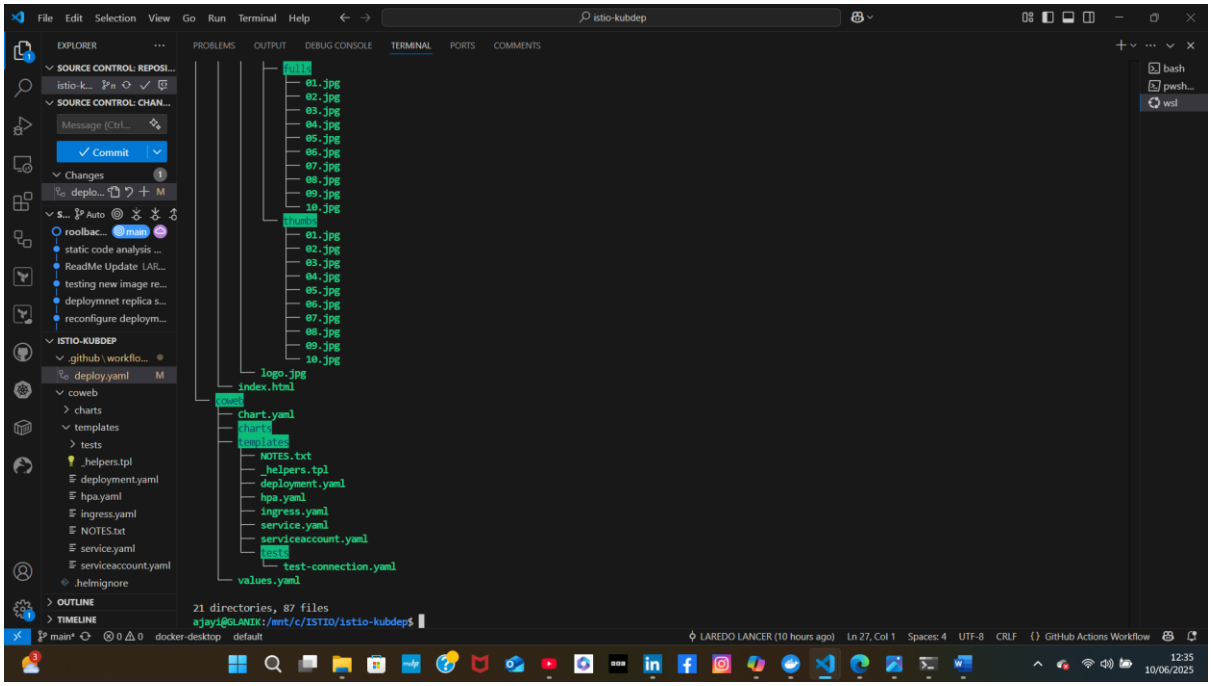
Deployment, service and pods



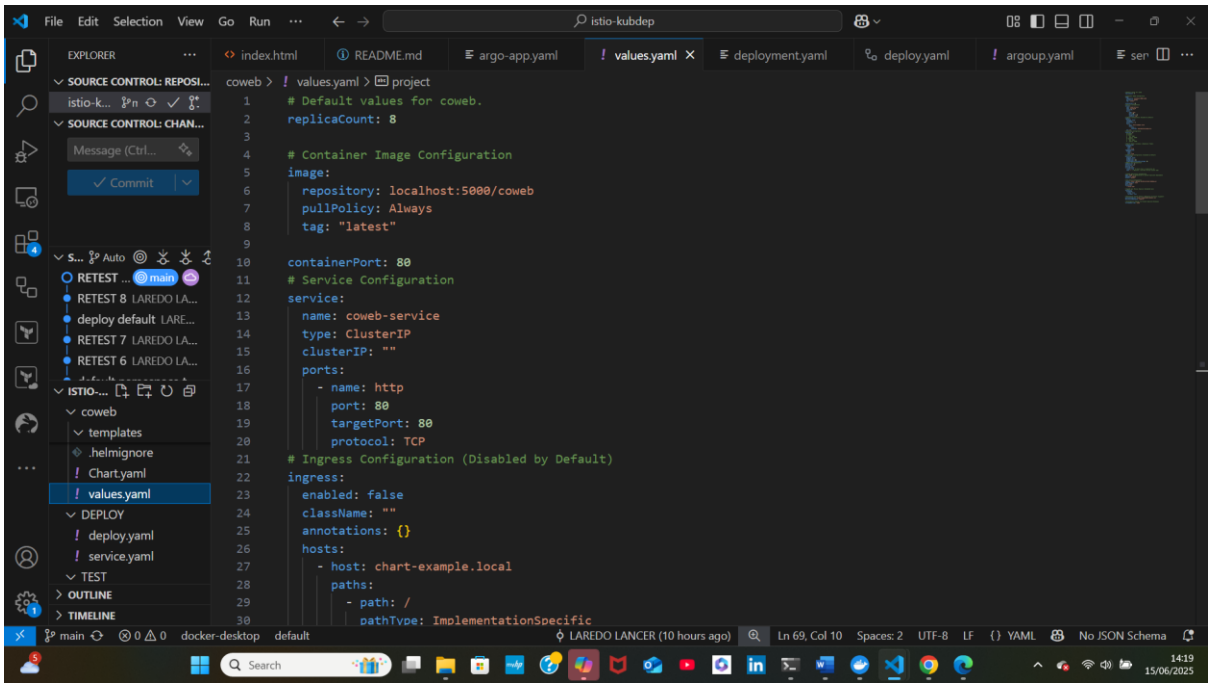
Helm Chart Tree



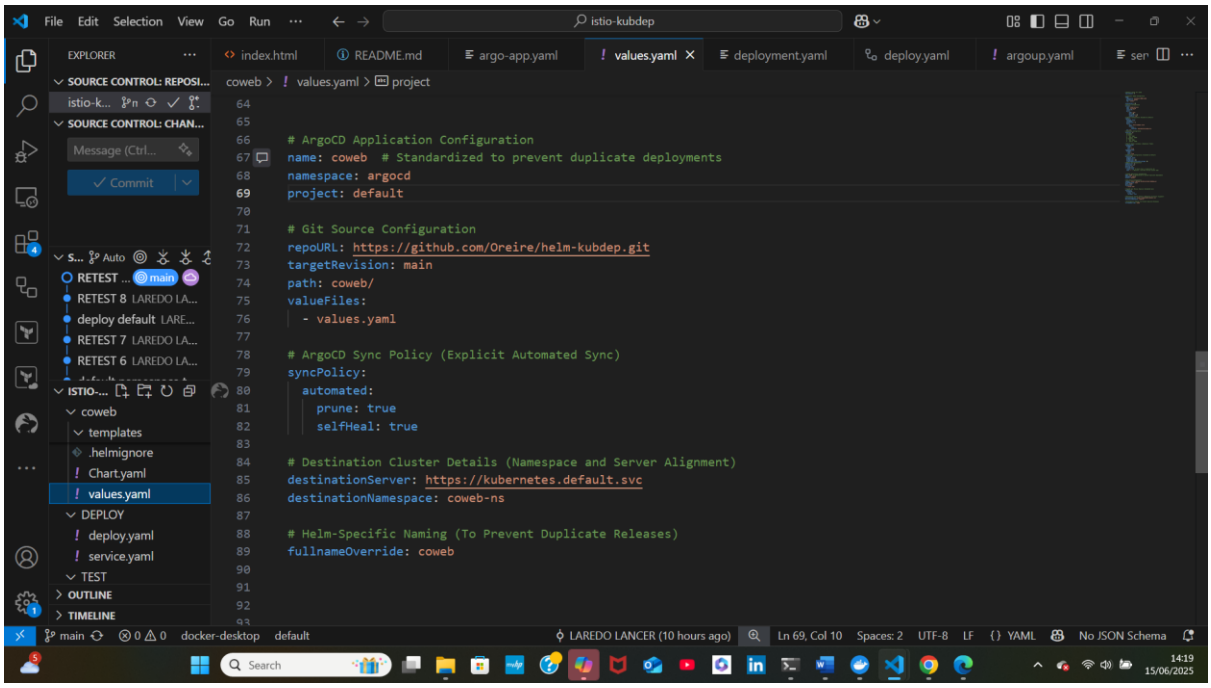




Helm coweb-service (Values.yaml)



ArgoCD Helm values.yaml



6.0 Conclusion

Organisations deploying containerised web applications on Kubernetes often rely on tools like GitHub Actions, CodeRabbit, Helm, and ArgoCD to enhance automation, efficiency, and governance across their development and deployment workflows. **GitHub Actions** streamlines CI/CD pipelines by automating builds, tests, and deployments triggered by code events. Moreover, its integration with repositories and support for reusable workflows ensure consistent and reproducible processes across environments.

Building on this automation layer, CodeRabbit introduces AI-powered code reviews that help maintain high code quality and accelerate development. By identifying potential issues and vulnerabilities early, it enhances application stability while enforcing best practices and strengthening compliance efforts.

Meanwhile, Helm simplifies Kubernetes application management through standardised, reusable charts, ensuring efficient deployment and configuration. Additionally, it facilitates scalable operations by enabling version control, parameterised deployments, and seamless rollbacks, improving reliability across different stages and environments.

To complete the pipeline, ArgoCD leverages a GitOps-based approach to deployment management, maintaining the desired state of applications in Git. As a result, it enables declarative, consistent, and auditable control over deployments while minimising configuration drift and supporting multi-cluster scalability.

Together, these tools create a fully integrated DevOps ecosystem that reduces manual intervention, increases release velocity, and ensures resilient, secure, and scalable delivery of modern web applications on Kubernetes.

7.0 Next-Steps

To optimize this CI/CD workflow for production readiness, the following enhancements are under considerations:

- **Integrate DevSecOps practices** to strengthen security and manage secrets more effectively throughout the pipeline.
- **Introduce automated testing stages**, including unit, integration, and Kubernetes smoke tests, to validate application behavior prior to deployment.
- **Implement monitoring and observability** using Istio to gain insight into service performance, traffic flow, and runtime anomalies.